Mini-project in digital/computational pathology

Created by Pierre Cordier (DVM, PhD, Dipl.ECVP)
with the use of ChatGPT (GPT-5.6 Luna) - 11/10/2026

For your first experiment, I recommend **ResNet-18 pretrained on ImageNet**, using PyTorch and Google Colab. It is a straightforward way to learn how a neural network turns histology patches into numerical embeddings, without having to train a model.

One important distinction for your veterinary pathology project:

- **Goal now:** learn the embedding pipeline and visualize how patches relate to one another.
- **Goal later:** find a model pretrained specifically on histopathology, and evaluate whether it transfers to veterinary tissue.

ResNet-18 is a good teaching baseline, but it was trained on everyday images, not histology. Its embeddings may still be useful for experimentation, but do not assume they capture diagnostically meaningful morphology.

1. What model are we using?

We'll use the standard pretrained ResNet-18 available directly through `torchvision`.

Property	Choice
Model	ResNet-18
Pretraining	ImageNet
Input	RGB patch, 224 × 224 pixels
Output	512 numbers per patch
Training	None; inference only
Hardware	Google Colab CPU or GPU

Why 512 numbers? The usual ResNet-18 classifier produces 1,000 ImageNet class scores. Instead, we'll remove the final classification layer and use the 512-dimensional feature vector immediately before it. This is our **embedding**.

For 500 patches, the result will be a matrix with shape `(500, 512)`: one row per patch.

2. Set up the notebook in Google Colab

Create a new notebook and run these cells in order. You do not need to install PyTorch separately in a standard Colab runtime.

First, check your runtime and install the required image-processing libraries:

```
import torch
import torchvision
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from PIL import Image
from pathlib import Path

print("PyTorch:", torch.__version__)
print("Torchvision:", torchvision.__version__)
print("Device:", "GPU" if torch.cuda.is_available() else "CPU")
```

We'll use `torchvision`'s official pretrained weights and their matching preprocessing transforms.

3. Load the pretrained encoder

```
import torch
from torchvision.models import resnet18, ResNet18_Weights
```

```
# Load pretrained ImageNet weights
weights = ResNet18_Weights.DEFAULT
model = resnet18(weights=weights)

# Remove the final 1000-class classification layer
model.fc = torch.nn.Identity()

# Prepare for inference, not training
model.eval()

# Use GPU if available, otherwise CPU
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = model.to(device)

# Use the preprocessing designed for these weights
preprocess = weights.transforms()

print("Device:", device)
print("Model ready")
```

What are the important lines?

• `weights=weights` loads learned parameters; without it, you'd have a randomly initialized network.

• `model.fc = torch.nn.Identity()` replaces the final classification layer with an identity operation, exposing the 512 features instead of 1,000 class scores.

• `model.eval()` sets inference behavior, which matters for layers such as batch normalization.

• `preprocess` resizes and normalizes images in the way expected by these weights.

You are not training the network here. You're using its existing learned parameters to describe each patch numerically.

4. Test it on one patch first

Use a patch you already saved from your WSI. For example, if your patch is at `results/patches/patch_001.png`, load it like this:

```
patch_path = "/content/results/patches/patch_001.png"

patch = Image.open(patch_path).convert("RGB")

plt.figure(figsize=(5, 5))
plt.imshow(patch)
plt.axis("off");
```

Change `patch_path` to the actual path in your Colab session. If you haven't extracted patches yet, you can upload a single representative H&E patch using Colab's Files panel.

Now generate its embedding:

```
# Convert the image to a normalized tensor
image_tensor = preprocess(patch)

# Add a batch dimension: (3, 224, 224) -> (1, 3, 224, 224)
batch = image_tensor.unsqueeze(0).to(device)

with torch.inference_mode():
    embedding = model(batch)

embedding = embedding.cpu().numpy()

print("Embedding shape:", embedding.shape)
print("First 10 values:", embedding[0, :10])
```

Expected shape:

```
Embedding shape: (1, 512)
```

The values are not probabilities and do not directly mean “lymphocyte,” “necrosis,” or “tumor.” They are learned numerical features. Their usefulness must be tested against your pathology task.

5. Generate embeddings for all your patches

Once the single-patch test works, process a folder of patches. For the first experiment, use perhaps 50–200 patches rather than the entire slide.

Assume your patches are saved as individual PNG or JPEG files in `/content/patches/`.

```
from pathlib import Path
from PIL import Image
import numpy as np
import pandas as pd
import torch

patch_dir = Path("/content/patches")

image_paths = sorted([
    p for p in patch_dir.iterdir()
    if p.suffix.lower() in {".png", ".jpg", ".jpeg"}
])

print("Number of patches:", len(image_paths))

if len(image_paths) == 0:
    raise ValueError("No images found. Check patch_dir.")
```

Now process the patches in batches. Batching is more efficient than calling the model separately for every image.

```
from torch.utils.data import Dataset, DataLoader

class PatchDataset(Dataset):
    def __init__(self, paths, transform):
        self.paths = paths
        self.transform = transform

    def __len__(self):
        return len(self.paths)

    def __getitem__(self, index):
        path = self.paths[index]

        with Image.open(path) as image:
            image = image.convert("RGB")
            tensor = self.transform(image)

        return tensor, str(path)


dataset = PatchDataset(image_paths, preprocess)

loader = DataLoader(
    dataset,
    batch_size=32,
    shuffle=False,
    num_workers=0
)

all_embeddings = []
all_paths = []

with torch.inference_mode():
    for images, paths in loader:
        images = images.to(device)

        features = model(images)

        all_embeddings.append(features.cpu().numpy())
        all_paths.extend(paths)

embeddings = np.concatenate(all_embeddings, axis=0)

print("Embedding matrix:", embeddings.shape)
```

For 100 patches, you should see:

```
Embedding matrix: (100, 512)
```

Save your results so you don't have to run the encoder again:

```
output_dir = Path("/content/embedding_results")
output_dir.mkdir(parents=True, exist_ok=True)

np.save(output_dir / "embeddings.npy", embeddings)

metadata = pd.DataFrame({
    "patch_path": all_paths,
    "embedding_row": np.arange(len(all_paths))
})

metadata.to_csv(output_dir / "patch_metadata.csv", index=False)

print("Saved embeddings and metadata.")
```

Important: this metadata only stores the image paths and row numbers. For your eventual WSI map, retain the original patch coordinates (`x`, `y`, level, patch size and slide ID) in the same metadata table. The embedding row must remain linked to its patch.

6. Visualize the embeddings with PCA

Now we can ask whether patches with similar numerical representations tend to group together.

```
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler

# Reduce 512 features to 2 dimensions
pca = PCA(n_components=2)
embedding_2d = pca.fit_transform(embeddings)

print("PCA shape:", embedding_2d.shape)
print(
    "Variance explained by two components:",
    round(pca.explained_variance_ratio_.sum(), 3)
)

plt.figure(figsize=(8, 6))
plt.scatter(
    embedding_2d[:, 0],
    embedding_2d[:, 1],
    s=20,
    alpha=0.75
)
plt.xlabel("Principal component 1")
plt.ylabel("Principal component 2")
plt.title("ResNet-18 patch embeddings")
plt.show()
```

PCA is a dimensionality-reduction method: it compresses the 512 numbers into two coordinates for plotting. The two axes are not biological measurements, and nearby points are not automatically the same tissue type.

To connect the plot to histology, inspect the patches represented by different points. For example, do patches that appear to contain collagen-rich stroma group together? Do background, folds or blood-rich areas form separate groups? These are questions to investigate, not outcomes to assume.

7. Make the plot interpretable by inspecting patches

Add the PCA coordinates to the metadata:

```
metadata["PC1"] = embedding_2d[:, 0]
metadata["PC2"] = embedding_2d[:, 1]
```

```
metadata.to_csv(
    output_dir / "patch_metadata_with_pca.csv",
    index=False
)
```

Clicking individual points is not necessary for this first experiment. Instead, plot the points and inspect selected patches by their row numbers:

```
# Change these indices to inspect different points
indices_to_inspect = [0, 10, 20, 30]

fig, axes = plt.subplots(
    1, len(indices_to_inspect),
    figsize=(4 * len(indices_to_inspect), 4)
)

for ax, idx in zip(axes, indices_to_inspect):
    with Image.open(all_paths[idx]) as image:
        ax.imshow(image.convert("RGB"))

    ax.set_title(f"Patch {idx}")
    ax.axis("off")

plt.tight_layout()
plt.show()
```

Try inspecting patches from different parts of the scatter plot rather than only consecutive indices.

8. What should you conclude?

Use these checks before moving on:

- The encoder runs on one patch.
- The output has 512 features per patch.
- You can process a folder of patches in batches.
- The embeddings and patch metadata are saved.
- You can visualize the embeddings using PCA.
- You have inspected actual patches from different parts of the plot.

One caution for your veterinary pathology work: **a visually convincing PCA plot is not proof that the encoder understands histopathology.** An ImageNet encoder may organize patches according to color, texture, scanner characteristics or artifacts rather than diagnostically meaningful morphology.

9. What comes next?

After this baseline works, I would compare it with a **histopathology-pretrained encoder**, using the same patches and preserving the same metadata. TIAToolbox provides a starting point for pathology-specific pretrained models, although model choice, availability and licensing should be checked for the particular weights you select.

That comparison will help you answer a more interesting question: do representations learned from histopathology organize your veterinary tissue patches more meaningfully than representations learned from everyday images?

For now, get the ResNet-18 pipeline working end to end. It gives you a simple, reproducible baseline before introducing more specialized models.